Tremol G03 CU Connection Guide

This guide is for the engineering and Armco IT teams preparing the Zoho Amarco middleware to communicate with the Tremol G03 fiscal control unit (CU) through Tremol ZFPLabServer.

Current integration status

The middleware has a client foundation for four read-only ZFPLabServer commands: `ReadStatus()`, `ReadVersion()`, `ReadCurrentReceiptInfo()`, and `ReadLastAndTotalReceiptNum()`. The Express application does not currently instantiate that client or expose CU diagnostics over its API. The application has no verified receipt-issuance flow. Do not treat network connectivity or a successful Zoho invoice mapping as evidence that the CU is ready to fiscalize.

Before connecting

Coordinate with the CU owner and Armco IT. Record the following information in the project handoff log. Each item prevents a specific connection or fiscal-data problem:

- **CU model, serial number, firmware, operating mode, service status, and location:** confirms the middleware is being connected to the intended unit and the correct protocol/mode.
- **Gateway PC hostname and fixed LAN address; ZFPLabServer version and service status:** identifies the host and software that must accept the middleware connection.
- **Confirmed CU connection from the gateway and approved network path from the middleware host:** separates a CU-to-gateway fault from a middleware-to-gateway network fault.
- **CU date/time, VAT categories and rates, invoice-number range, and item settings:** records fiscal configuration that must be checked before any receipt operation.
- **Vendor documentation for the installed version and approved read-only commands:** avoids using URL syntax or commands that do not match the installed software.
- **Named operator, test window, test invoice/account, and recovery procedure:** ensures someone authorized can supervise the check and respond safely if device state changes.

Do not put credentials or private customer data in this guide, source control, command output, or the handoff log.

Network path

The expected path is:

```
Middleware host -- approved private LAN/VPN --> Gateway PC / ZFPLabServer --> Tremol G03 CU
```

The CU site has confirmed that ZFPLabServer is using port `8000`. Verify the port and command URL format against the installed version's vendor documentation before use. Do not expose this plain HTTP service to the public internet. Use only an Armco-approved private network or secured gateway route.

Why this step matters: ZFPLabServer is the bridge between the middleware and the locally connected CU. Verifying the real gateway and private route first avoids mistaking a network problem for a CU or invoice-mapping problem, while limiting access protects a service that does not provide public-internet transport security.

From an authorized PowerShell session, check that the gateway is reachable. Replace the example with the address supplied by Armco IT:

```
$gatewayIp = 'GATEWAY_IP'
Test-NetConnection $gatewayIp -Port 8000
```

Why: this checks only whether a TCP connection to the gateway's service port is possible. It does not send a CU command or prove the CU is ready, so it is a low-impact first network check.

If the TCP check fails, ask Armco IT to check the VPN route, host firewall, and ZFPLabServer listener. Do not change firewall rules or expose the port independently.

Read-only connection check

First run the checks on the gateway PC itself. Confirm that the ZFPLabServer test page opens and use the vendor-approved URLs. The project runbook currently records this example:

```
$zfp = 'http://localhost:8000'
(Invoke-WebRequest -Uri "$zfp/ReadStatus()" -TimeoutSec 10).Content
(Invoke-WebRequest -Uri "$zfp/ReadVersion()" -TimeoutSec 10).Content
(Invoke-WebRequest -Uri "$zfp/ReadLastAndTotalReceiptNum()" -TimeoutSec 10).Content
```

Why: starting on the gateway removes the VPN/LAN route as a variable. If this local check fails, troubleshoot ZFPLabServer or the CU connection before involving the middleware host.

Run one command at a time and wait for its response. **Why:** this makes it clear which command produced a result and avoids overlapping requests to the device. Save the timestamp, command name, sanitized result code, and outcome so the operator can correlate the result with device logs and reproduce the check. Do not share unredacted response data outside the approved engineering channel. If the CU reports an error or a value is unexpected, stop and consult the operator/vendor; guessing at recovery commands could change CU state.

Repeat from the middleware host only after Armco IT has approved network access. **Why:** this verifies the actual route the deployed service will use, after local device communication is known to work. `localhost` on that host refers to the middleware computer, not the gateway. Use the gateway address in place of `localhost` only if the network route and URL format have been confirmed.

On-site procedure: first visit

Follow these steps in order with the CU operator present. This procedure verifies the connection and records the CU's starting state; it does not issue a receipt.

1. **Meet the named CU operator and confirm the approved test window.** Ask the operator to confirm that the CU is available for read-only checks and that no one is in the middle of a sale or service operation. **Why:** the operator knows the live state of the unit and can stop the check if it would interfere with business activity.
2. **Identify the CU and gateway.** Compare the CU model/serial number with the unit Armco intended for the integration. Record the gateway hostname, LAN address, ZFPLabServer version, and the person responsible for the gateway. **Why:** this prevents testing the wrong unit or configuring the middleware against an unintended host.
3. **Ask the operator to confirm the CU is in its normal idle/ready state.** Do not press keys, open/close a receipt, alter configuration, or disconnect cables as part of this check. **Why:** preserving the existing state avoids interrupting operations or changing fiscal settings.
4. **On the gateway PC, confirm ZFPLabServer is running and the vendor test page responds.** Use the installed version's manual to confirm its local URL and command syntax. **Why:** proving the local service is available isolates CU/gateway issues before testing the network path.

5. **Run the read-only commands locally, one at a time:** `ReadStatus()`, `ReadVersion()`, `ReadCurrentReceiptInfo()`, then `ReadLastAndTotalReceiptNum()`. Use the URLs approved for the installed server version. Record each timestamp, HTTP status, result code, and sanitized response. **Why:** these checks establish device identity, readiness, any current receipt state, and the starting receipt counters without asking the CU to issue a receipt.
6. **Stop if a current receipt is open, status is not ready, a command errors, or a value is unexpected.** Show the result to the operator and contact the vendor/Armco technical owner for interpretation. **Why:** an open or uncertain state needs an authorized operator's assessment; attempting to clear it could affect fiscal records.
7. **Only after local checks pass, verify the approved route from the middleware host.** First run `Test-NetConnection <gateway-address> -Port 8000`; if approved and reachable, repeat only the same read-only commands using the confirmed gateway URL. **Why:** this verifies the end-to-end route the middleware will use while preserving the known-good local baseline.
8. **Review and sign off the handoff record with the operator and IT owner.** Note which checks passed, any blocked checks, and who owns the next action. **Why:** the next engineer can distinguish verified facts from assumptions and avoid repeating disruptive troubleshooting.

At the end of this visit, a successful outcome means only that the approved read-only checks reached the intended CU and returned expected values. It does not authorize receipt issuance, establish invoice/tax mapping, or prove KRA/eTIMS compliance. Do not run a live sale or receipt test unless a separate written test plan identifies the test data, operator, expected fiscal effect, and recovery/reconciliation procedure.

What to expect

Network check

When the route and listener are available, `Test-NetConnection` should show:

```
ComputerName     : <gateway address>
RemotePort       : 8000
TcpTestSucceeded : True
```

`TcpTestSucceeded : False` means the host/port could not be reached. It does not identify whether the cause is VPN routing, a firewall, the wrong address/port, or a stopped service; give the result to Armco IT to investigate.

Read command response

For a successful request, expect HTTP `200` and XML with a `Res` root element whose numeric `Code` is `0`. For example:

```
<Res Code="0"></Res>
```

The CU/ZFPLabServer may include command-specific child values. A response can look like this shape:

```
<Res Code="0">
  <Res Name="Example" Value="123" Type="String" />
</Res>
```

These XML snippets show the envelope accepted by the middleware client; they are not guaranteed literal output for each command. Confirm the meaning, units, and expected values for `ReadStatus()`, `ReadVersion()`, and receipt counters with the installed ZFPLabServer manual and the CU operator. In particular, compare the last receipt number before and after any separately authorized device test.

Common results and next action

Result	Meaning	What to do and why
TCP check succeeds, HTTP `200`, XML `Code="0"`	The gateway answered and the command reported success.	Record the command and result; have the operator confirm the returned CU values are expected.
TCP check fails or request times out	The host/port did not respond in time.	Stop and ask Armco IT to check routing, firewall, address, and service status. A timeout does not prove the CU did nothing; never retry a receipt command blindly.
HTTP error such as `404` or `503`	The server responded, but the request path or service was rejected/unavailable.	Verify the installed server version, URL/command syntax, and service status with IT/vendor.
HTTP `200` with nonzero XML code, e.g. `<Res Code="40">...`	HTTP worked, but ZFPLabServer/CU reported a command error.	Preserve the sanitized code/message and ask the operator/vendor to interpret it; do not improvise recovery commands.
HTTP `200` with malformed XML, missing `Res`, or missing/nonnumeric `Code`	The response does not match the protocol envelope this client expects.	Save a sanitized sample and verify protocol/version compatibility before proceeding.

The middleware client treats any nonzero CU result code as an error and expects the XML envelope above. ZFPLabServer-specific error codes and status fields must be interpreted using the vendor documentation; this guide does not assign meanings to them.

Middleware setup and current limits

The current environment schema has no ZFPLabServer URL setting, and `src/server.ts` does not wire the CU client into the application. Therefore, there is no supported middleware API endpoint for CU diagnostics yet. Do not assume that setting a URL in `.env` or calling the Zoho endpoints will communicate with the CU.

The `TremolZfpLabServerClient` can be used by application code with a base URL and timeout, but the client is an infrastructure component, not a deployed endpoint. It accepts only the four read commands listed above, expects a successful XML response containing a numeric `Res` code, and serializes commands within one client instance.

Before adding an authenticated diagnostics endpoint, implementation work should:

1. **Add validated configuration for the private ZFPLabServer base URL and request timeout.** This makes the target explicit per environment and bounds how long an unavailable gateway can hold a request.
2. **Instantiate the client in server composition and inject it through an explicit dependency.** This connects the existing client to the running application without hiding device access in unrelated route logic.
3. **Expose a read-only diagnostics route protected by operator authentication/authorization.** This gives authorized staff a supported way to check the CU while preventing arbitrary callers from querying the device.
4. **Return only the fields needed by operators and sanitize device errors and logs.** This supports troubleshooting while reducing exposure of raw device/customer data.
5. **Verify behavior with the exact installed ZFPLabServer and CU firmware in a controlled environment.** This catches version and protocol differences before the middleware is relied on operationally.

Receipt issuance and fiscal safety

Receipt-issuing commands are intentionally absent from the current client. Before implementing them, obtain and verify the correct vendor protocol/SDK, payload fields, VAT mapping, rounding rules, receipt lifecycle, cancellation behavior, and responses for the installed CU configuration. **Why:** these define what the CU will print and how an operation can be safely completed or reconciled; assumptions here can create incorrect or duplicate fiscal receipts. Confirm customer PIN, item codes, units, currencies, discounts, credit notes, and invoice-state handling with the

appropriate technical and compliance owners because each can affect fiscal treatment or whether an invoice is eligible.

Never automatically retry a command after a timeout or lost response when a receipt might have been issued. Reconcile the CU's current/last receipt state first, because the CU may have completed the operation even if the middleware did not receive its response. Persist a receipt identifier only after confirmed success to avoid recording an unissued receipt. CU connectivity alone does not establish KRA/eTIMS compliance; validate that integration separately because device communication and regulatory reporting are distinct outcomes.

Connection handoff record

Date/time	Host and network path	ZFPLabServer version	Command	HTTP/result code	Outcome and evidence	Owner/next action

Do not include passwords, tokens, private keys, or unrestricted invoice/customer data in this record.

Related documentation

- Armco G03 Remaining Work Runbook — local development checks, gateway checklist, implementation stages, and go/no-go criteria.
- Project README — application setup, API authentication, and current project status.